

COMSATS University Islamabad, Vehari Campus

Information Communication Technology (CSC101)

Lab 09

Topic: Problem Solving with Functions

E-mail: mariamsherwani434@gmail.com

HOW TO STUDY THESE NOTES

Each activity includes:

1. **Exact Problem Statement** (what the question asks)
 2. **Thinking Approach** (how to break the problem)
 3. **Step-by-Step Solution** (no steps skipped)
-

1. ACTIVITY 1 – TEMPERATURE CONVERSION

The following flowchart consists of two subroutines. User will input the choice “C” if he/she wants to convert temperature from Fahrenheit to Celsius and “F” otherwise. Depending on the user’s choice subroutine will be called and convert the temperature into required unit.

EXACT PROBLEM STATEMENT

The user enters a choice:

- “C” → Convert temperature from **Fahrenheit to Celsius**
- “F” → Convert temperature from **Celsius to Fahrenheit**

The program must: Call the correct function (subroutine) based on user choice.

THINKING APPROACH

- Main program should **NOT do calculations**
- It should only **decide which function to call**
- Each function performs one task

STEP-BY-STEP SOLUTION

MAIN PROGRAM

1. Start
2. Ask user: Enter choice (C/F)
3. IF choice == "C" → Call ConvertToCelsius()
4. ELSE IF choice == "F" → Call ConvertToFahrenheit()
5. End

FUNCTION: ConvertToCelsius

Formula:

$$C = (F - 32) \times 5 / 9$$

Steps:

1. Input Fahrenheit value (F)
2. Apply formula
3. Display Celsius value

FUNCTION: ConvertToFahrenheit

Formula:

$$F = (C \times 9 / 5) + 32$$

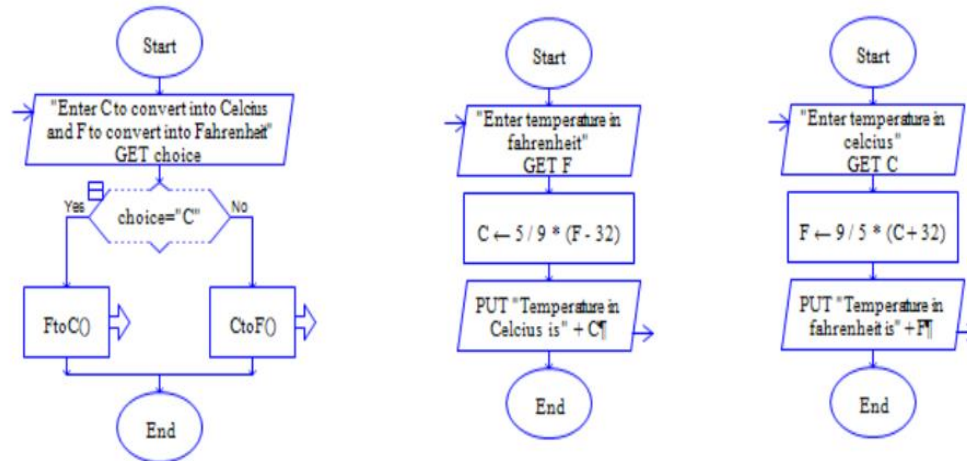
Steps:

1. Input Celsius value (C)
2. Apply formula
3. Display Fahrenheit value

KEY IDEA

Main program = decision maker Functions = work performers

FLOWCHART



2. ACTIVITY 2 – FUNCTION CALLING ANOTHER FUNCTION

Following flowchart illustrate the call of a function from another function. The main will call a function CheckEven(n). This function will again call the function CheckPrime(n) if the passed argument is odd and print the corresponding ,message otherwise.

EXACT PROBLEM STATEMENT

Main program calls CheckEven(n)

CheckEven(n):

- If number is EVEN → print message
- If number is ODD → call CheckPrime(n)

THINKING APPROACH

- First check EVEN/ODD
- If EVEN → stop
- If ODD → go deeper (check prime)

STEP-BY-STEP SOLUTION

MAIN PROGRAM

1. Start
2. Input number n
3. Call CheckEven(n)
4. End

FUNCTION: CheckEven(n)

Logic:

IF $n \% 2 == 0$

 PRINT "Even Number"

ELSE

 CALL CheckPrime(n)

FUNCTION: CheckPrime(n)

Definition of Prime:

A number is prime if it has only 2 factors: 1 and itself.

Steps:

1. IF $n \leq 1 \rightarrow$ Not Prime
2. Loop i from 2 to n-1
3. IF $n \% i == 0 \rightarrow$ Not Prime
4. If loop finishes $\rightarrow$ Prime

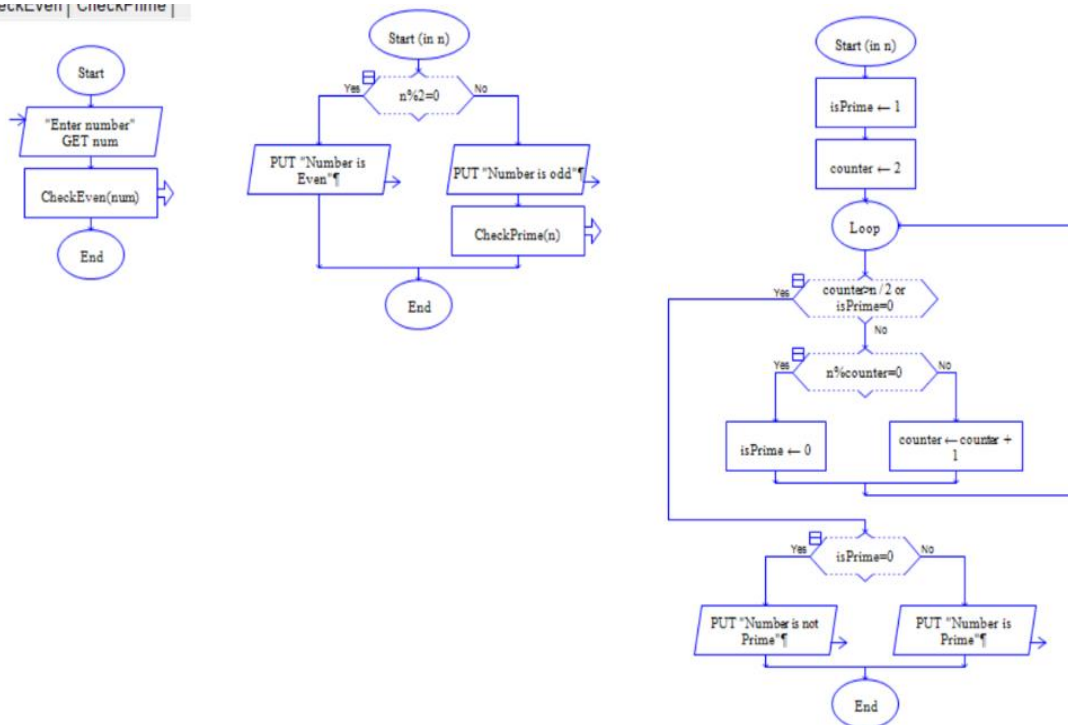
EXAMPLE

Input: 9

- 9 is odd $\rightarrow$ CheckPrime(9)
- $9 \% 3 == 0 \rightarrow$ Not Prime

KEY IDEA

[CheckEven | CheckPrime]



3. LAB TASK – CREDIT CARD VALIDATION (LUHN ALGORITHM)

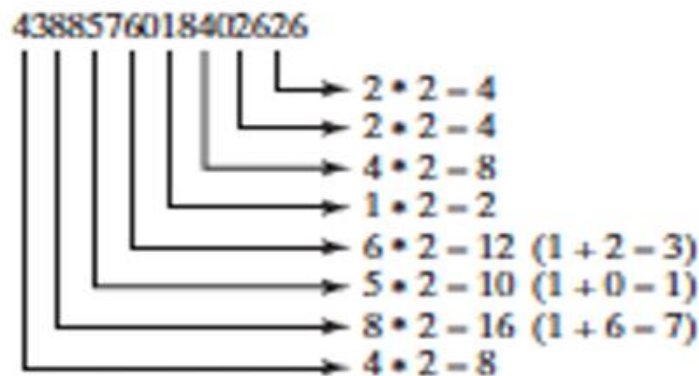
Credit card numbers follow certain patterns: It must have between 13 and 16 digits, and the number must start with:

- 4 for Visa cards
- 5 for MasterCard credit cards

- 37 for American Express cards
- 6 for Discover cards

The Luhn's algorithm is useful to determine whether a card number is entered correctly or whether a credit card is scanned correctly by a scanner. Credit card numbers are generated following this validity check, which can be described as follows (for illustration consider the card number 4388576018402626):

1. Double every second digit from right to left. If doubling of a digit results in a two-digit number, add up the two digits to get a single digit number.
2. Now add all single-digit numbers from Step 1. $4 + 4 + 8 + 2 + 3 + 1 + 7 + 8 = 37$
3. Add all digits in the odd places from right to left in the card number. $6 + 6 + 0 + 8 + 0 + 7 + 8 + 3 = 38$
4. Sum the results from Steps 2 and 3 $37 + 38 = 75$
5. If the result from Step 4 is divisible by 10, the card number is valid; otherwise, it is invalid.



For example, the number 4388576018402626 is invalid, but the number 4388576018410707 is valid. Draw a flowchart that prompts the user to enter a credit card number as an integer. Display whether the number is valid or invalid. Design your flowchart to use the following functions/subroutines:

Return true if the card number is valid

def isValid(number):

Get the result from Step 2

def sumOfDoubleEvenPlace(number):

```
# Return this number if it is a single digit, otherwise, return the sum of the two digits
def getDigit(number):

# Return sum of odd place digits in number
def sumOfOddPlace(number):

# Return true if the digit d is a prefix for number
def prefixMatched(number, d):

# Return the number of digits in d
def getSize(d):

# Return the first k number of digits from number. If the number of digits in number is less
than k, return number.
def getPrefix(number, k):
```

EXACT PROBLEM STATEMENT

The user enters a credit card number.

The program must check:

1. Length is between 13 and 16 digits
2. Prefix (starting digits) is valid
3. Apply Luhn Algorithm
4. Output: VALID or INVALID

WHAT IS A PREFIX?

Prefix means the starting digits of a number

Example:

- Number: 438857...
- Prefix = 4
- Number: 378282...
- Prefix = 37

👉 Prefix helps identify the **card type**

WHAT DO 4, 5, 37, 6 REPRESENT?

These are **card company identifiers**:

- 4 → Visa Card
- 5 → MasterCard
- 37 → American Express (AMEX)
- 6 → Discover Card

This means:

- If number starts with 4 → it is Visa
- If number starts with 37 → it is AMEX

THINKING APPROACH

Big problem → divide into small functions

That is why we use multiple functions.

LUHN ALGORITHM (VERY SIMPLE STEPS)

STEP 1

From RIGHT side:

- Take every second digit
- Double it
- If result is 2-digit → add digits

Example: $8 \times 2 = 16 \rightarrow 1+6 = 7$

STEP 2

Add all results from Step 1

STEP 3

Add remaining digits (odd positions)

STEP 4

Add both sums

STEP 5

IF total % 10 == 0 → VALID

ELSE → INVALID

FUNCTIONS

isValid(number)

- Final decision function

sumOfDoubleEvenPlace(number)

- Processes even-position digits

getDigit(number)

- Converts double-digit into single digit
- Example: 16 → 1+6 = 7

sumOfOddPlace(number)

- Adds odd-position digits

prefixMatched(number, d)

- Checks if number starts with valid prefix

getSize(number)

- Counts total digits

getPrefix(number, k)

- Extracts first k digits
- Example: 123456 → first 2 digits = 12

COMPLETE PROGRAM FLOW

1. Input card number
2. Check length (13–16)
3. Check prefix (4,5,37,6)
4. Calculate even sum
5. Calculate odd sum
6. Add both
7. Check divisible by 10
8. Print VALID / INVALID

FULL EXAMPLE

Number: 4388576018410707

Even sum = 37

Odd sum = 38 Total = 75

$75 \% 10 \neq 0 \rightarrow$ INVALID

Solved Example for understanding:

Card Number:

4 3 8 8 5 7 6 0 1 8 4 1 0 7 0 7

Count from right Side

Digit	Position (right→left)	Odd/Even
7	1	Odd
0	2	Even
7	3	Odd
0	4	Even
1	5	Odd
4	6	Even
8	7	Odd
1	8	Even
6	9	Odd
7	10	Even
5	11	Odd
8	12	Even
8	13	Odd
3	14	Even

4	15	Odd
4	16	Even

Step 1: Even positions digits (right se 2,4,6...)

Even positions: 2,4,6,8,10,12,14,16

Digits = 0,0,4,1,7,8,3,4

Step 2: Double the even position digits

- $0 \rightarrow 0 \times 2 = 0$
- $0 \rightarrow 0 \times 2 = 0$
- $4 \rightarrow 4 \times 2 = 8$
- $1 \rightarrow 1 \times 2 = 2$
- $7 \rightarrow 7 \times 2 = 14 \rightarrow 1+4 = 5$
- $8 \rightarrow 8 \times 2 = 16 \rightarrow 1+6 = 7$
- $3 \rightarrow 3 \times 2 = 6$
- $4 \rightarrow 4 \times 2 = 8$

Sum of doubled even digits:

$$0 + 0 + 8 + 2 + 5 + 7 + 6 + 8 = 36$$

Even sum = 36

Step 3: Odd position digits (right se 1,3,5...)

Odd positions: 1,3,5,7,9,11,13,15

Digits = 7,7,1,8,6,5,8,4

Sum of odd digits:

$$7 + 7 + 1 + 8 + 6 + 5 + 8 + 4 = 46$$

Odd sum = 46

Step 4: Total sum

$$\text{Even sum} + \text{Odd sum} = 36 + 46 = 82$$

Step 5: Divisible by 10 check

$$82 \% 10 = 2 \rightarrow \text{Not divisible by 10}$$

Result

Card number 4388576018410707 → INVALID

FINAL UNDERSTANDING

- Prefix = starting digits (card type)
- Functions = small tasks
- Main = control logic
- Luhn = validation check

Number Validation with Luhn Algorithm

